

## **JOBSHEET VII SEARCHING**

---



Disusun oleh:

Nama : Galih Candra Kirana  
No. Absen : 10  
Kelas : TI 1G  
NIM : 254107020080

**POLITEKNIK NEGERI MALANG**  
Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141  
**TAHUN 2025-2026**

## 7.1 Tujuan Praktikum

Setelah melakukan materi praktikum ini, mahasiswa mampu:

- 1 Menjelaskan mengenai algoritma Searching.
- 2 Membuat dan mendeklarasikan struktur algoritma Searching.
- 3 Menerapkan dan mengimplementasikan algoritma Searching.

## 7.2 Searching/ Pencarian Menggunakan Algoritma *Sequential Search*

### 7.2.1 Langkah-langkah Percobaan

Class Mahasiswa:

```
package Jobsheet7;

public class Mahasiswa10 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    Mahasiswa10() {
    }

    Mahasiswa10(String nm, String name, String kls, double ip) {
        nim = nm;
        nama = name;
        ipk = ip;
        kelas = kls;
    }

    void tampilInformasi() {
        System.out.println("Nama: " + nama);
        System.out.println("NIM: " + nim);
        System.out.println("Kelas: " + kelas);
        System.out.println("IPK: " + ipk);
    }
}
```

```
}
```

Class MahasiswaBerprestasi:

```
package Jobsheet7;

public class MahasiswaBerprestasi10 {

    Mahasiswa10[] listMhs;
    int idx;

    MahasiswaBerprestasi10(int jml) {
        listMhs = new Mahasiswa10[jml];
    }

    void tambah(Mahasiswa10 m) {
        if (idx < listMhs.length) {
            listMhs[idx] = m;
            idx++;
        } else {
            System.out.println("Data sudah penuh");
        }
    }

    void tampil() {
        for (Mahasiswa10 m : listMhs) {
            m.tampilInformasi();
            System.out.println("-----");
        }
    }

    void bubbleSort() {
        for (int i = 0; i < listMhs.length - 1; i++) {
```

```

        for (int j = 1; j < listMhs.length - i; j++) {
            if (listMhs[j].ipk > listMhs[j - 1].ipk) {
                Mahasiswa10 tmp = listMhs[j];
                listMhs[j] = listMhs[j - 1];
                listMhs[j - 1] = tmp;
            }
        }
    }
}

```

```

void selectionSort() {
    for (int i = 0; i < listMhs.length - 1; i++) {
        int idxMin = i;
        for (int j = i + 1; j < listMhs.length; j++) {
            if (listMhs[j].ipk < listMhs[idxMin].ipk) {
                idxMin = j;
            }
        }

        Mahasiswa10 tmp = listMhs[idxMin];
        listMhs[idxMin] = listMhs[i];
        listMhs[i] = tmp;
    }
}

```

```

void insertionSort() {
    for (int i = 1; i < listMhs.length; i++) {
        Mahasiswa10 temp = listMhs[i];
        int j = i;

        while (j > 0 && listMhs[j - 1].ipk > temp.ipk) {
            listMhs[j] = listMhs[j - 1];

```

```

        j--;
    }

    listMhs[j] = temp;
}
}

int sequentialSearch(double cari) {
    int posisi = -1;
    for (int j = 0; j < listMhs.length; j++) {
        if (listMhs[j].ipk == cari) {
            posisi = j;
            break;
        }
    }
    return posisi;
}

void tampilPosisi(double x, int pos) {
    if (pos != -1) {
        System.out.println("Data mahasiswa dengan IPK : " + x + " ditemukan
pada indeks " + pos);
    } else {
        System.out.println("Data " + x + " tidak ditemukan");
    }
}

void tampilDataSearch(double x, int pos) {
    if (pos != -1) {
        System.out.println("nim\t : " + listMhs[pos].nim);
        System.out.println("nama\t : " + listMhs[pos].nama);
        System.out.println("kelas\t : " + listMhs[pos].kelas);
    }
}

```

```

        System.out.println("ipk\t : " + x);
    } else {
        System.out.println("Data mahasiswa dengan IPK " + x + " tidak
ditemukan");
    }
}
}
}

```

Class MahasiswaDemo:

```

package Jobsheet7;

import java.util.Scanner;

public class MahasiswaDemo10 {

    static void inputData(Scanner sc, MahasiswaBerprestasi10 list, int jumMhs)
    {

        for (int i = 0; i < jumMhs; i++) {

            System.out.println("Data mahasiswa ke-" + (i + 1) + ": ");

            Mahasiswa10 m = new Mahasiswa10();

            System.out.print("Masukkan NIM : ");
            m.nim = sc.nextLine();
            System.out.print("Masukkan Nama : ");
            m.nama = sc.nextLine();
            System.out.print("Masukkan Kelas : ");
            m.kelas = sc.nextLine();
            System.out.print("Masukkan IPK : ");
            m.ipk = sc.nextDouble();
            sc.nextLine();
            System.out.println("-----");
        }
    }
}

```

```

        list.tambah(m);
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int jumMhs = 5;

    MahasiswaBerprestasi10 list = new MahasiswaBerprestasi10(jumMhs);

    inputData(sc, list, jumMhs);

    list.tampil();

    // melakukan pencarian data sequential
    System.out.println("-----");
    System.out.println("Pencarian data");
    System.out.println("-----");
    System.out.println("Masukkan ipk mahasiswa yang dicari: ");
    System.out.print("IPK: ");
    double cari = sc.nextDouble();

    System.out.println("menggunakan sequential searching");
    double posisi = list.sequentialSearch(cari);
    int pss = (int) posisi;
    list.tampilPosisi(cari, pss);
    list.tampilDataSearch(cari, pss);
    sc.close();
}
}

```

## 7.2.2 Verifikasi Hasil Percobaan

```
Data mahasiswa ke-1:
Masukkan NIM : 111
Masukkan Nama : adi
Masukkan Kelas : 2
Masukkan IPK : 3.6
-----
Data mahasiswa ke-2:
Masukkan NIM : 222
Masukkan Nama : tio
Masukkan Kelas : 2
Masukkan IPK : 3.8
-----
Data mahasiswa ke-3:
Masukkan NIM : 333
Masukkan Nama : ila
Masukkan Kelas : 2
Masukkan IPK : 3.0
-----
Data mahasiswa ke-4:
Masukkan NIM : 444
Masukkan Nama : lia
Masukkan Kelas : 2
Masukkan IPK : 3.5
-----
Data mahasiswa ke-5:
Masukkan NIM : 555
Masukkan Nama : fia
Masukkan Kelas : 2
Masukkan IPK : 3.3
-----

Nama: adi
NIM: 111
Kelas: 2
IPK: 3.6
-----
Nama: tio
NIM: 222
Kelas: 2
IPK: 3.8
-----
Nama: ila
NIM: 333
Kelas: 2
IPK: 3.0
-----
Nama: lia
NIM: 444
Kelas: 2
IPK: 3.5
-----
Nama: fia
NIM: 555
Kelas: 2
IPK: 3.3
-----

Pencarian data
-----
Masukkan ipk mahasiswa yang dicari:
IPK: 3.5
menggunakan sequential searching
Data mahasiswa dengan IPK :3.5 ditemukan pada indeks 3
nim : 444
nama : lia
kelas : 2
ipk : 3.5
```

## 7.2.3 Pertanyaan!

1. Jelaskan perbedaan metod **tampilDataSearch** dan **tampilPosisi** pada class **MahasiswaBerprestasi**!  
Jawaban:  
Perbedaan pada kedua method tersebut adalah pada method tampilData(), jika pencarian ipk pada data mahasiswa ditemukan, method tersebut hanya akan mencetak indeks data mahasiswa, dan jika tidak ditemukan, maka akan mencetak tidak ditemukan. Sedangkan method tampilDataSearch() menampilkan seluruh data mahasiswa jika ditemukan pada pencarian ipk yang sama, dan mencetak tidak ditemukan jika ipk pencarian tidak ada pada data mahasiswa.
2. Jelaskan fungsi **break** pada kode program di bawah ini!  
Jawaban:  
Fungsi break pada kode tersebut berfungsi untuk menghentikan perulangan lebih awal jika data pencarian ditemukan sebelum perulangan selesai.
3. Apa fungsi variabel pos atau indeks hasil pencarian dalam program sequential search?  
Jawaban:  
Fungsi variabel pos adalah untuk menyimpan indeks hasil dari pencarian sequential search.
4. Jika terdapat lebih dari satu data dengan nilai yang sama, hasil pencarian sequential search yang dibuat di atas akan menampilkan data ke berapa? Jelaskan.  
Jawaban:



Hasilnya akan menampilkan data yang pertama kali cocok dengan kata kuncinya(ditemukan), karena di kode program terdapat kondisi break jika data sudah ditemukan saat pertama kali.

5. Berkaitan dengan pertanyaan nomor 2 di atas, apa yang terjadi jika perintah **break** dihapus dari kode di atas?

Jawaban:

Jika break dihapus, maka ketika ada kata kunci yang sama tetap akan melakukan perulangan sampai selesai. Jika ada data lagi yang sama dengan kata kunci, maka hasil indeks nya akan menempa indeks awal yang sama tadi.

## **7.3 Searching/ Pencarian Menggunakan Algoritma *Binary Search***

### **7.3.1 Langkah – langkah percobaan**

Class MahasiswaBerprestasi:

```
package Jobsheet7;

public class Mahasiswa10 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    Mahasiswa10() {
    }

    Mahasiswa10(String nm, String name, String kls, double ip) {
        nim = nm;
        nama = name;
        ipk = ip;
        kelas = kls;
    }

    void tampilInformasi() {
        System.out.println("Nama: " + nama);
        System.out.println("NIM: " + nim);
    }
}
```

```

        System.out.println("Kelas: " + kelas);
        System.out.println("IPK:  " + ipk);
    }
}

```

Class MahasiswaBerprestasi:

```

package Jobsheet7;

public class MahasiswaBerprestasi10 {

    Mahasiswa10[] listMhs;
    int idx;

    MahasiswaBerprestasi10(int jml) {
        listMhs = new Mahasiswa10[jml];
    }

    void tambah(Mahasiswa10 m) {
        if (idx < listMhs.length) {
            listMhs[idx] = m;
            idx++;
        } else {
            System.out.println("Data sudah penuh");
        }
    }

    void tampil() {
        for (Mahasiswa10 m : listMhs) {
            m.tampilInformasi();
            System.out.println("-----");
        }
    }
}

```

```
    }  
}
```

```
void bubbleSort() {  
    for (int i = 0; i < listMhs.length - 1; i++) {  
        for (int j = 1; j < listMhs.length - i; j++) {  
            if (listMhs[j].ipk > listMhs[j - 1].ipk) {  
                Mahasiswa10 tmp = listMhs[j];  
                listMhs[j] = listMhs[j - 1];  
                listMhs[j - 1] = tmp;  
            }  
        }  
    }  
}
```

```
void selectionSort() {  
    for (int i = 0; i < listMhs.length - 1; i++) {  
        int idxMin = i;  
        for (int j = i + 1; j < listMhs.length; j++) {  
            if (listMhs[j].ipk < listMhs[idxMin].ipk) {  
                idxMin = j;  
            }  
        }  
  
        Mahasiswa10 tmp = listMhs[idxMin];  
        listMhs[idxMin] = listMhs[i];  
        listMhs[i] = tmp;  
    }  
}
```

```

void insertionSort() {
    for (int i = 1; i < listMhs.length; i++) {
        Mahasiswa10 temp = listMhs[i];
        int j = i;

        while (j > 0 && listMhs[j - 1].ipk > temp.ipk) {
            listMhs[j] = listMhs[j - 1];
            j--;
        }

        listMhs[j] = temp;
    }
}

```

```

int sequentialSearch(double cari) {
    int posisi = -1;
    for (int j = 0; j < listMhs.length; j++) {
        if (listMhs[j].ipk == cari) {
            posisi = j;
            break;
        }
    }
    return posisi;
}

```

```

void tampilPosisi(double x, int pos) {
    if (pos != -1) {
        System.out.println("Data mahasiswa dengan IPK : " + x + " ditemukan
pada indeks " + pos);
    } else {
        System.out.println("Data " + x + " tidak ditemukan");
    }
}

```

```

    }
}

void tampilDataSearch(double x, int pos) {
    if (pos != -1) {
        System.out.println("nim\t : " + listMhs[pos].nim);
        System.out.println("nama\t : " + listMhs[pos].nama);
        System.out.println("kelas\t : " + listMhs[pos].kelas);
        System.out.println("ipk\t : " + x);
    } else {
        System.out.println("Data mahasiswa dengan IPK " + x + " tidak
ditemukan");
    }
}

int findBinarySearch(double cari, int left, int right) {
    int mid;
    if (right >= left) {
        mid = (left + right) / 2;
        if (cari == listMhs[mid].ipk) {
            return (mid);
        } else if (listMhs[mid].ipk > cari) {
            return findBinarySearch(cari, left, mid - 1);
        } else
            return findBinarySearch(cari, mid + 1, right);
    }
    return -1;
}
}

```

Class MahasiswaDemo:

```

package Jobsheet7;

import java.util.Scanner;

public class MahasiswaDemo10 {

    static void inputData(Scanner sc, MahasiswaBerprestasi10 list, int jumMhs)
    {

        for (int i = 0; i < jumMhs; i++) {

            System.out.println("Data mahasiswa ke-" + (i + 1) + ": ");

            Mahasiswa10 m = new Mahasiswa10();

            System.out.print("Masukkan NIM  : ");
            m.nim = sc.nextLine();
            System.out.print("Masukkan Nama  : ");
            m.nama = sc.nextLine();
            System.out.print("Masukkan Kelas : ");
            m.kelas = sc.nextLine();
            System.out.print("Masukkan IPK  : ");
            m.ipk = sc.nextDouble();
            sc.nextLine();
            System.out.println("-----");
            list.tambah(m);
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int jumMhs = 5;
    }
}

```

```
MahasiswaBerprestasi10 list = new MahasiswaBerprestasi10(jumMhs);
```

```
inputData(sc, list, jumMhs);
```

```
list.tampil();
```

```
// melakukan pencarian data Sequential
```

```
// System.out.println("-----");
```

```
// System.out.println("Pencarian data");
```

```
// System.out.println("-----");
```

```
// System.out.println("Masukkan ipk mahasiswa yang dicari: ");
```

```
// System.out.print("IPK: ");
```

```
// double cari = sc.nextDouble();
```

```
// System.out.println("menggunakan sequential searching");
```

```
// double posisi = list.sequentialSearch(cari);
```

```
// int pss = (int) posisi;
```

```
// list.tampilPosisi(cari, pss);
```

```
// list.tampilDataSearch(cari, pss);
```

```
// melakukan pencarian data Binary
```

```
System.out.println("-----");
```

```
System.out.println("Pencarian data");
```

```
System.out.println("-----");
```

```
System.out.println("Masukkan ipk mahasiswa yang dicari: ");
```

```
System.out.print("IPK: ");
```

```
double cari2 = sc.nextDouble();
```

```
System.out.println("-----");
```

```
System.out.println("menggunakan binary search");
```

```

        System.out.println("-----");

        double posisi2 = list.findBinarySearch(cari2, 0, jumMhs - 1);

        int pss2 = (int) posisi2;

        list.tampilPosisi(cari2, pss2);

        list.tampilDataSearch(cari2, pss2);

        sc.close();

    }

}

```

### 7.3.2 Verifikasi Hasil Percobaan

```

Data mahasiswa ke-1:
Masukkan NIM : 111
Masukkan Nama : adi
Masukkan Kelas : 2
Masukkan IPK : 3.1
-----
Data mahasiswa ke-2:
Masukkan NIM : 222
Masukkan Nama : ila
Masukkan Kelas : 2
Masukkan IPK : 3.2
-----
Data mahasiswa ke-3:
Masukkan NIM : 333
Masukkan Nama : lia
Masukkan Kelas : 2
Masukkan IPK : 3.3
-----
Data mahasiswa ke-4:
Masukkan NIM : 444
Masukkan Nama : susi
Masukkan Kelas : 2
Masukkan IPK : 3.5
-----
Data mahasiswa ke-5:
Masukkan NIM : 555
Masukkan Nama : anita
Masukkan Kelas : 2
Masukkan IPK : 3.7
-----

```

```

Nama: adi
NIM: 111
Kelas: 2
IPK: 3.1
-----
Nama: ila
NIM: 222
Kelas: 2
IPK: 3.2
-----
Nama: lia
NIM: 333
Kelas: 2
IPK: 3.3
-----
Nama: susi
NIM: 444
Kelas: 2
IPK: 3.5
-----
Nama: anita
NIM: 555
Kelas: 2
IPK: 3.7
-----
Pencarian data
-----
Masukkan ipk mahasiswa yang dicari:
IPK: 3.7
-----
menggunakan binary search
-----
Data mahasiswa dengan IPK :3.7 ditemukan pada indeks 4
nim : 555
nama : anita
kelas : 2
ipk : 3.7

```

### 7.3.3 Pertanyaan!

1. Tunjukkan pada kode program yang mana proses *divide* dijalankan!  
Jawaban:  
$$\text{mid} = (\text{left} + \text{right}) / 2;$$
2. Tunjukkan pada kode program yang mana proses *conquer* dijalankan!  
Jawaban:

```

if (cari == listMhs[mid].ipk) {
    return (mid);
} else if (listMhs[mid].ipk > cari) {
    return findBinarySearch(cari, left, mid - 1);
} else

```



```
return findBinarySearch(cari, mid + 1, right);
```

3. Apa fungsi left, right, dan mid?

Jawaban:

Fungsinya untuk memberi penanda, left untuk indeks paling kiri, mid untuk indeks tengah, right untuk indeks paling kanan.

4. Jika data IPK yang dimasukkan tidakurut. Apakah program masih dapat berjalan? Mengapa demikian?

Jawaban:

Masih bisa jalan, tetapi hasilnya nanti akan jadi salah yang kemungkinan akan mereturn data tidak ditemukan karena faktor data yang tidak urut akan menyebabkan pencarian menskip nilai yang di cari.

5. Jika IPK yang dimasukkan dari IPK terbesar ke terkecil (misal: 3.8, 3.7, 3.5, 3.4, 3.2) dan elemen yang dicari adalah 3.2. Bagaimana hasil dari binary search? Apakah sesuai? Jika tidak sesuai maka ubahlah kode program *binary seach* agar hasilnya sesuai

Jawaban:

Hasilnya pasti tidak ditemukan karena areanya menggeser ke kiri bukan ke kanan.

Modifikasi:

```
else if (listMhs[mid].ipk < cari) {  
    return findBinarySearch(cari, left, mid - 1);  
} else  
return findBinarySearch(cari, mid + 1, right);
```

6. Jelaskan bagaimana *binary search* menentukan bahwa data yang dicari tidak ditemukan di dalam array.

Jawaban:

Data yang dicari tidak ditemukan, jika pada pencocokan nilai ipk indeks tengah dengan data ipk mahasiswa tidak sama sampai perulangan berakhir.

7. Modifikasi program di atas yang mana jumlah mahasiswa yang diinputkan sesuai dengan masukan dari keyboard.

Jawaban:

Class MahasiswaDemo:

```
package Jobsheet7;
```

```
import java.util.Scanner;
```

```
public class MahasiswaDemo10 {
```

```
    static void inputData(Scanner sc, MahasiswaBerprestasi10 list, int jumMhs)  
    {
```

```
        for (int i = 0; i < jumMhs; i++) {  
            System.out.println("Data mahasiswa ke-" + (i + 1) + ": ");
```

```
            Mahasiswa10 m = new Mahasiswa10();
```

```

        System.out.print("Masukkan NIM  : ");
        m.nim = sc.nextLine();
        System.out.print("Masukkan Nama  : ");
        m.nama = sc.nextLine();
        System.out.print("Masukkan Kelas : ");
        m.kelas = sc.nextLine();
        System.out.print("Masukkan IPK  : ");
        m.ipk = sc.nextDouble();
        sc.nextLine();
        System.out.println("-----");
        list.tambah(m);
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Masukkan jumlah mahasiswa: ");
    int jumMhs = sc.nextInt();
    sc.nextLine();

    MahasiswaBerprestasi10 list = new MahasiswaBerprestasi10(jumMhs);

    inputData(sc, list, jumMhs);

    list.tampil();

    // melakukan pencarian data Sequential
    // System.out.println("-----");
    // System.out.println("Pencarian data");
    // System.out.println("-----");
    // System.out.print("Masukkan ipk mahasiswa yang dicari: ");
    // System.out.print("IPK: ");
    // double cari = sc.nextDouble();

    // System.out.println("menggunakan sequential searching");
    // double posisi = list.sequentialSearch(cari);
    // int pss = (int) posisi;
    // list.tampilPosisi(cari, pss);
    // list.tampilDataSearch(cari, pss);

    // melakukan pencarian data Binary
    System.out.println("-----");
    System.out.println("Pencarian data");
    System.out.println("-----");
    System.out.print("Masukkan ipk mahasiswa yang dicari: ");
    System.out.print("IPK: ");
    double cari2 = sc.nextDouble();
    System.out.println("-----");

```

```
        System.out.println("menggunakan binary search");
        System.out.println("-----");
        double posisi2 = list.findBinarySearch(cari2, 0, jumMhs - 1);
        int pss2 = (int) posisi2;
        list.tampilPosisi(cari2, pss2);
        list.tampilDataSearch(cari2, pss2);
        sc.close();
    }
}
```